Chapter 20
Numerical Procedures

Chapter 13 opened by saying that its tree was a teaching device rather than a computational method, and that how to build trees efficiently, how many steps they need, and how to make them converge belonged here. Chapter 15 said the same of the free boundary in an American option, Chapter 14 of the choice between two ways to discretize a process, and Chapter 17 of the simulation needed to value a relative-performance award; the smile-fitting models of Section 19.4, local volatility above all, have no formula and are computed the same way. This chapter answers those questions.

The subject is narrow and the stakes are not. Every price in Chapters 13 through 19 that does not come from a closed-form formula is produced by one of the three methods in this chapter, and the difference between a method that converges and one that merely appears to is the difference between a book of trades that is valued correctly and one that is not.

Section 20.1 says what numerical methods are for. Sections 20.2 to 20.5 are trees in practice: the control variate, trinomial trees, and dividends. Sections 20.6 to 20.10 are Monte Carlo: discretizing the process, variance reduction, several assets, and Greeks from a simulation. Section 20.11 is American Monte Carlo and Section 20.12 finite differences; Section 20.13 chooses among the three methods, and Section 20.14 records where they came from.

20.1 WHAT NUMERICAL METHODS ARE FOR

A closed-form price exists only when three things happen to be true at once: the payoff is simple, the process is simple, and there is no early exercise. Break any one and a number must be computed.

Table 20.1: Why a problem needs a numerical method, and which one suits it.

Feature	Example	Natural method
Early exercise	American put, Bermudan swaption	Tree or finite difference
Path dependence	Asian, lookback, barrier	Monte Carlo
Many state variables	Basket, quanto basket, CVA	Monte Carlo
Both at once	Callable structured note	American Monte Carlo
Greeks across a grid	A whole book, revalued	Finite difference

Table 20.1 is the chapter in one page, and the pattern in it has a reason. [TN15] showed that the pricing partial differential equation and the risk-neutral expectation are the same statement (the Feynman–Kac relation), so a price can be obtained either by solving the equation backward or by simulating the expectation forward. The two directions have opposite strengths, and everything in Table 20.1 follows from that:

Backward methods (trees, finite differences) sweep from expiration to today and therefore know, at every node, the value of continuing. That is exactly what an early-exercise test needs. They also produce the value at every state, so the Greeks come out of the same computation.

Their weakness is dimension: a grid with m points along each of d state variables has m^d nodes (a million at $m = 100$ and $d = 3$), which is fatal beyond about three.

Forward methods (Monte Carlo) generate whole paths and average. **Path dependence**, a payoff that depends on the route and not only the destination, therefore costs no more to evaluate than a terminal payoff does: the whole path has already been generated, and an Asian or a lookback payoff simply reads it. The cost also grows linearly rather than exponentially in the number of state variables. Their weakness is the mirror image: at any intermediate date a simulated path does not know its own continuation value, which is why early exercise is hard and why Section 20.11 exists.

Transform methods are a fourth route, and a narrow one. Where a model's characteristic function (the Fourier transform of its terminal distribution) is known in closed form (Heston, Bates and the jump models of Chapter 21), a European price is a single integral over it, which the fast Fourier transform of Carr and Madan (1999) or the cosine expansion of Fang and Oosterlee (2008) evaluates fast enough for Chapter 21 to calibrate Heston to a whole surface inside an optimizer. Having no early exercise and no path dependence, the route supplements the other three rather than replacing any.

Convergence is the whole subject. Every method here produces an answer for any settings at all; the question is always whether that answer is close, and how one would know. The chapter's recurring warning is that a number that looks stable can be wrong, and that the cheapest defense is to compute something whose answer is already known.

20.2 BINOMIAL TREES IN PRACTICE

Chapter 13 built the tree and Section 13.9 observed that its value settles to a limit at a rate of about $1/n$, oscillating between odd and even n. Here is that statement made precise, because both halves of it matter.

Take a European put with $S_0 = K = 50$, $r = 5\%$, $\sigma = 30\%$, and six months to expiration. The Black–Scholes–Merton value is 3.582934.

Table 20.2: Cox–Ross–Rubinstein tree against the Black–Scholes–Merton value.

n	Tree	Error	n	Tree	Error
5	3.789185	+0.206251	100	3.572449	−0.010485
10	3.479418	−0.103516	101	3.592980	+0.010046
11	3.675951	+0.093017	200	3.577688	−0.005246
20	3.530789	−0.052145	201	3.587979	+0.005045
21	3.631453	+0.048519	500	3.580835	−0.002099
50	3.561990	−0.020943	1000	3.581884	−0.001050

Two features of Table 20.2 deserve attention.

The error halves when n doubles. From 100 to 200 to 500 to 1000 the error runs −0.0105, −0.0052, −0.0021, −0.0011. This is first-order convergence, and it is bad news: to gain one decimal place you must take ten times as many steps, and the work at n steps is of order n^2 nodes, written $O(n^2)$, so a factor of ten in accuracy costs a factor of a hundred in time.

The sign alternates with the parity of n. Even n undershoots, odd n overshoots, and they do so by almost the same amount. The reason is geometric: with $S_0 = K$, an even number of steps places a terminal node exactly at the strike while an odd number straddles it, so the discretized distribution sits differently against the payoff's kink in the two cases.

The alternation is a property of this contract, not of trees. What sets the error is where the strike falls between two terminal nodes, and a node lands on it exactly when $\kappa + n$ is an even

Wulin Suo: WSuo@Queensu.ca. For Personal Use, Do Not Distribute!

integer, for $\kappa = \ln(K/S_0)/(\sigma\sqrt{\Delta t})$. At the money $\kappa = 0$ and that reads "n even." Otherwise κ grows like $\sqrt{n}$, the strike drifts through the lattice as steps are added, and the signs stop alternating: at $K = 55$ the errors at $n = 26, 27, 28$ run -0.006, $+0.026$, $+0.003$. Section 20.3 turns this into a rule for choosing n at any strike.

How many steps. There is no universal answer, but the shape of Table 20.2 gives the discipline. Never report a tree value without computing it at two step counts. If 200 and 400 steps disagree in the third decimal, the third decimal is not real. Dealers' pricing systems typically run 200 to 1000 steps for a vanilla American option and use the correction of the next section to do better with fewer.

20.3 THE CONTROL VARIATE

This section treats one technique, the main repair for the slow, oscillating convergence Section 13.9 deferred: it reaches far beyond trees, and the rest of the chapter keeps returning to it.

The idea is general. Suppose you want A, which you can only estimate, and there is a related quantity E that you can estimate *by the same method* and whose true value E^{true} you already know. Then

$$\boxed{\hat{A}_{\text{corrected}} = \hat{A} + \left(E^{\text{true}} - \hat{E}\right)} \tag{20.1}$$

where $\hat{A}$ is the method's estimate of the target, $\hat{E}$ its estimate of the companion (the same method at the same settings, on the same tree or the same paths), E^{true} the companion's known exact value, and $\hat{A}_{\text{corrected}}$ the number reported. The correction is the method's own error on a problem where the error is visible.

For an American option the natural companion is the European option with the same strike and expiration: price both on the same tree, and correct the American value by the tree's error on the European, which the Black–Scholes–Merton formula gives exactly.

Example 20.1 The control variate on an American put

Take the same contract, now American: $S_0 = K = 50$, $r = 5\%$, $\sigma = 30\%$, six months. Its value is 3.69702, computed two ways that share no arithmetic (corrected trees at thousands of steps, and the finite-difference grid of Section 20.12), against the European 3.582934, an early-exercise premium of 0.114.

n	Plain tree	Error	Corrected	Error
50	3.68552	−0.01150	3.70646	+0.00944
51	3.71691	+0.01989	3.69700	−0.00002
100	3.69135	−0.00567	3.70183	+0.00481
101	3.70717	+0.01015	3.69712	+0.00010
200	3.69419	−0.00283	3.69944	+0.00242
201	3.70213	+0.00511	3.69709	+0.00007

On odd step counts the correction is extraordinary: at $n = 201$ the error falls from +0.00511 to +0.00007, a factor of about 75, for roughly twice the work. On even step counts it changes the sign of the error and shrinks it by less than a fifth.

The asymmetry says exactly what the control variate does. With $S_0 = K$ and odd n, no node sits at the strike, and the American and the European tree carry the same large error from straddling the payoff kink—a *shared* error, which (20.1) removes completely. With even n a node sits exactly on the strike and the two errors part company: the European's is as large as before, but the American's is only about half of it, because the American value also carries the error in locating

Wulin Suo: WSuo@Queensu.ca. For Personal Use, Do Not Distribute!

the early-exercise boundary, which the European option does not have and therefore cannot correct.

A control variate removes what the two problems share and nothing else. The practical corollary is to choose a companion problem as close to the target as possible.

The step count is the second lever, and it works at any strike. Nothing in that explanation mentions parity: what decides the matter is whether a node sits on the strike, which by Section 20.2 happens when $\kappa + n$ is even and is furthest from happening when $\kappa + n$ is odd. At the money $\kappa = 0$ and that reads "take n odd," which is Section 20.2's alternation turned into an instruction. Away from the money it reads "choose n so that $\kappa + n$ is close to an odd integer," whatever the parity of n.

At $K = 55$, step counts that leave the strike near a midpoint are about sixteen times more accurate for the same work.

Notice also, in Figure 20.1, what the correction does to the shape of the sequence. The plain tree oscillates, so extrapolating across it is dangerous; the corrected sequence oscillates too, but its odd branch already sits on zero and its even branch decays smoothly toward it, so extrapolating along a branch is safe. Following the correction with **Richardson extrapolation** (taking $2V(2n) - V(n)$, which cancels an error of the form C/n provided both step counts carry the same C) is the standard recipe in practice, and it reaches four-decimal accuracy at a few hundred steps. The proviso is not automatic, because C depends on the strike's **offset**, its position between the two terminal nodes that bracket it: the two step counts must share an offset, which at the money means both even.

Other parameterizations remove the oscillation at its source. The equal-probability tree fixes $p = 1/2$ and puts the drift in u and d; Leisen and Reimer (1996) choose u, d and p by inverting the normal distribution so that the tree is centered on the strike, which makes convergence monotone and close to second order with no extrapolation. It is a common choice for vanilla American equity options.

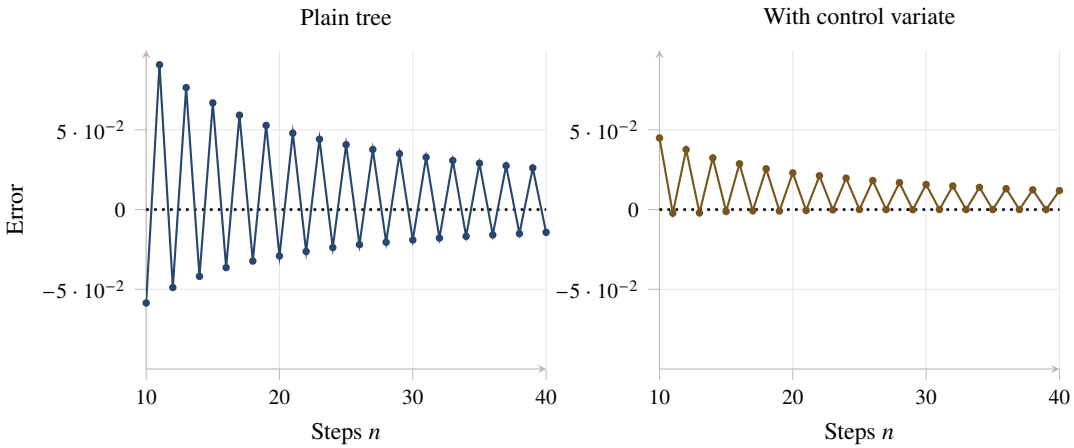


Figure 20.1: American put error against step count, same vertical scale.

20.4 TRINOMIAL TREES

Let the price move up, down, *or stay*. With the log-price $x = \ln S$ spaced Δx apart and drift $\nu = r - q - \sigma^2/2$,

$$p_u = \frac{1}{2}\left(\frac{\sigma^2\Delta t + \nu^2\Delta t^2}{\Delta x^2} + \frac{\nu\Delta t}{\Delta x}\right), \quad p_d = \frac{1}{2}\left(\frac{\sigma^2\Delta t + \nu^2\Delta t^2}{\Delta x^2} - \frac{\nu\Delta t}{\Delta x}\right), \quad p_m = 1 - p_u - p_d \tag{20.2}$$

Wulin Suo: WSuo@Queensu.ca. For Personal Use, Do Not Distribute!

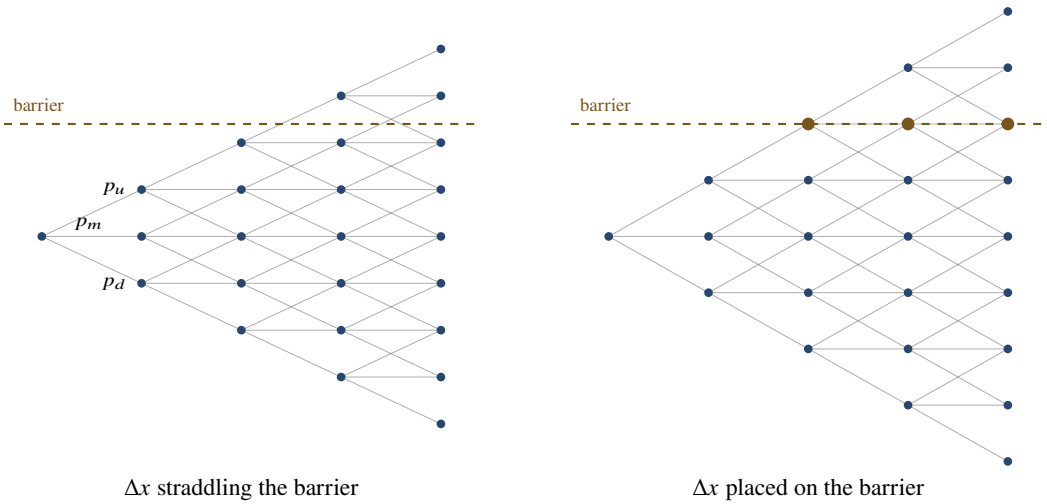
These are not a definition but a consequence. Over one step the change δx in the true log-price has

$$E[\delta x] = \nu\,\Delta t, \qquad E[\delta x^2] = \sigma^2\Delta t + \nu^2\Delta t^2$$

while the tree moves x by $+\Delta x$, 0 or $-\Delta x$ and so delivers $(p_u - p_d)\,\Delta x$ and $(p_u + p_d)\,\Delta x^2$. Setting each pair equal, together with $p_u + p_m + p_d = 1$, gives three equations in three unknowns, and (20.2) is their solution. The tree is built to reproduce the first two moments of the process it stands for, which is what Chapter 13's binomial tree does with two branches rather than three.

That leaves Δx free, and $\sigma\sqrt{3\Delta t}$ is the common choice: it puts $p_u + p_d$ at about $1/3$, so the middle branch carries roughly $2/3$ of the probability and the two outer ones stay safely positive.

Figure 20.2 shows the geometry, and the recombination is the property that matters: a step adds two nodes rather than tripling them, so the work grows with the square of the number of steps instead of with 3^n.



Δx straddling the barrier Δx placed on the barrier

Figure 20.2: Four steps of a trinomial lattice, drawn against the same barrier at two choices of Δx.

The extra branch buys two things and does *not* buy a third.

It does not converge faster. On the European put above, the trinomial errors at $n = 50, 100, 200$ are -0.0207, -0.0103, -0.0052, against binomial errors of -0.0209, -0.0105, -0.0052. Both are $O(1/n)$ and they are numerically almost identical. Anyone who reaches for a trinomial tree expecting accuracy for free will not get it.

It buys a free parameter. In (20.2), Δx and Δt are chosen independently, whereas the binomial tree ties them by $u = e^{\sigma\sqrt{\Delta t}}$. That freedom is what lets a node be placed exactly on a barrier, as the right panel of Figure 20.2 does and the left one fails to. Chapter 34 shows that a barrier option priced on a tree whose nodes straddle the barrier converges badly, because the barrier is effectively moved to the nearest node.

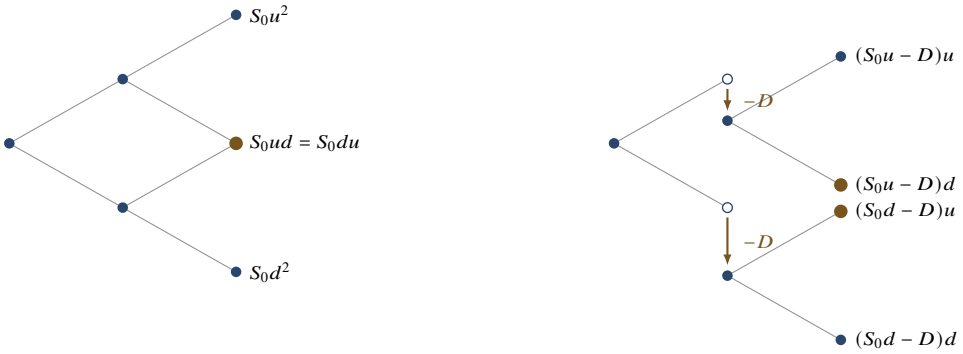
It buys room for a varying volatility. A local-volatility model in the sense of Section 19.4 needs a different σ at each node; in a binomial tree that changes u from node to node, and the paths stop meeting. The trinomial keeps Δx fixed and absorbs σ in the probabilities.

It is also a finite difference scheme. Section 20.12 shows that the explicit finite difference method *is* a trinomial tree, with the three probabilities appearing as the coefficients of the discretized derivatives. The two subjects are one subject.

Wulin Suo: WSuo@Queensu.ca. For Personal Use, Do Not Distribute!

20.5 DIVIDENDS AND THE NON-RECOMBINING TREE

Section 13.10 flagged this as the first place the tree's convenience breaks. Two of the three ways a stock can pay a dividend cost the tree nothing. A *continuous yield* is handled in the drift: replace r by $r - q$ and the geometry is untouched. A *discrete proportional* dividend (a known fraction δ of whatever the price turns out to be at the ex-date t_d) is free for a different reason: every node at that date is multiplied by the same $1 - \delta$, so the lattice is rescaled rather than distorted and recombines exactly as before. A known cash dividend is not free. If the price drops by \$2 on the ex-date, an up-then-down path and a down-then-up path no longer meet, because \$2 is subtracted at different price levels: subtracting a constant moves nodes by different proportions, where multiplying by one does not. Every node standing at the ex-date then starts a tree of its own, so each dividend multiplies the node count by the number of nodes at its date, and with several ex-dates the count grows geometrically until the tree is unusable. Figure 20.3 shows the difficulty.



No dividend: the paths meet

Cash dividend: they do not

Figure 20.3: Two steps around an ex-date, without and with a cash dividend D.

The standard remedy is the **displaced tree**, which splits the stock price into a part that behaves like a lognormal and a part that does not.

Write $S = S^* + De^{-r(t_d - t)}$ for $t < t_d$, where D is the dividend and t_d its date. The second term is a known, riskless amount, the present value of a certain payment. Build the tree on S^*, which has no dividend and therefore recombines, then add the present value of the dividend back at every node when computing the exercise decision and the payoff. One caveat goes with it: the σ on the tree is the volatility of S^*, so a volatility quoted on S must be scaled up to roughly $\sigma S/S^*$. Example 20.2 takes its 30% as the volatility of S^*.

Figure 20.4 draws the construction rather than its result. The dashed lattice is S^*: it is the tree that is actually built, it recombines, and it runs straight through the ex-date without noticing it. The solid nodes above it are what the exercise test compares against, S^* with the dividend's present value added back, and past the ex-date that value is zero and the two are one lattice. The recombination lives in S^* and the economics in S, which is the trick.

Example 20.2 A cash dividend done three ways

Give the American put of Example 20.1 a \$2.00 dividend at $t = 0.25$.

The displaced spot is $S^* = 50 - 2e^{-0.05\times0.25} = 48.0248$, and a tree built on it gives 4.6063, 4.6096 and 4.6100 at 100, 400 and 1,600 steps. That sequence looks settled and is not: the converged American put value is 4.6098, which the tree reaches only after tens of thousands of steps and the finite-difference method of Section 20.12 confirms. The corresponding European

Wulin Suo: WSuo@Queensu.ca. For Personal Use, Do Not Distribute!

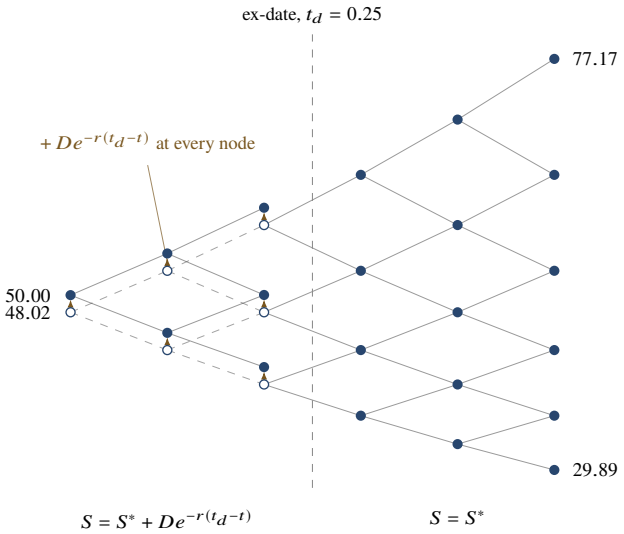


Figure 20.4: Five steps on the displaced tree of Example 20.2: dashed, S^*; solid, S^* plus the dividend's present value. $S_0 = 50$, $D = 2$ at $t_d = 0.25$, $\sigma = 30\%$, $r = 5\%$.

value is 4.4689 from the Black–Scholes–Merton formula on S^*.

Compare that with two shortcuts:

Treatment	Value	Error
Displaced tree (correct)	4.6098	—
Dividend ignored	3.6970	−19.8%
Spot reduced by the full \$2.00	4.6385	+0.6%

Ignoring the dividend is catastrophic and obviously so. Subtracting the undiscounted dividend instead of its present value is a small error here (\$0.03, or 0.6%), but it is the kind that scales: with a larger dividend, a longer time to the ex-date, or a higher rate, the discounting matters more, and the error is one-directional rather than random.

The early-exercise premium also moves. Without the dividend it was 0.114; with it, 4.6098 − 4.4689 = 0.141. The rise comes from the lower price, which leaves the put deeper in the money, and not from the dividend itself: on a non-paying stock at the same spot of 48.0248 the premium is 0.157. Before the ex-date a dividend makes early exercise of a *put* less attractive, not more: the put holder wants the stock price low, and the dividend does that for them without their having to exercise. That runs opposite to the call case of Section 11.10.

Holding the dividend *amount* fixed shows what its form and timing are worth. A 4% proportional dividend at t_d has a present value of exactly $\delta S_0 = \$2.00$, since $e^{-rt}S_t$ is a martingale, against \$1.9752 for the \$2.00 of cash, and the put is worth correspondingly more: 4.6223 against 4.6098. Spread the same 4% smoothly over the six months instead ($q = -\ln(0.96)/0.5 = 8.16\%$) and the terminal distribution is unchanged, $0.96S_0e^{(r-\sigma^2/2)T+\sigma W_T}$ either way, so the European value is 4.4810 under both; but the American falls to 4.4812: the early-exercise premium drops from 0.1414 to 0.0002, because a continuous yield lowers the drift to $r - q = -3.16\%$ everywhere, and against a stock expected to fall a put is worth holding rather than exercising. The form of the payment moves the value by 0.3%, its timing by more than 3%.

Wulin Suo: WSuo@Queensu.ca. For Personal Use, Do Not Distribute!

20.6 MONTE CARLO SIMULATION

Simulate the risk-neutral process forward, compute the payoff on each path, average, and discount:

$$\hat{f} = e^{-rT}\,\frac{1}{M}\sum_{i=1}^{M}\text{payoff}\big(\text{path}_i\big) \tag{20.3}$$

This is the definition of a risk-neutral price from Section 13.2, evaluated by brute force. Nothing about it is subtle, and its two properties follow immediately from the law of large numbers and the central limit theorem.

It is unbiased. Each sampled payoff has the price as its discounted expectation, and their average converges to it regardless of the payoff's shape, which is why a path-dependent payoff needs no special handling.

Its error falls like $1/\sqrt{M}$. Write $f_i = e^{-rT}\,\text{payoff}(\text{path}_i)$ for the discounted payoff on path i. The standard error of (20.3) is

$$\text{s.e.}(\hat{f}) = \frac{s}{\sqrt{M}}, \qquad s^2 = \frac{1}{M-1}\sum_{i=1}^{M}\left(f_i - \hat{f}\right)^2 \tag{20.4}$$

where s is the sample standard deviation of the M discounted payoffs and $\hat{f}$ is their average, which is (20.3) itself. Both come out of one pass over the paths, so a simulation reports its own accuracy, which nothing else in this chapter does. To gain one decimal place you must run a hundred times as many paths, and Example 20.3 shows what that costs.

Example 20.3 The cost of a decimal place

Simulate the European put of Table 20.2, whose exact value is 3.582934:

Paths	Estimate	Standard error	Actual error
1,000	3.71346	0.16249	+0.13053
10,000	3.63958	0.05062	+0.05664
100,000	3.58181	0.01595	−0.00113
1,000,000	3.58916	0.00504	+0.00623

The standard error falls by $\sqrt{10} \approx 3.16$ for every tenfold increase in paths, which is the $1/\sqrt{M}$ rule holding to within 2%. The actual errors bounce around inside it (at a million paths the estimate happens to be worse than at a hundred thousand), which is the point of a standard error: it is a statement about the distribution of the answer, not about this answer.

A million paths bought a standard error of half a cent on an option worth $3.58. A 200-step binomial tree, costing a small fraction of the time, was already inside a cent. For a vanilla European option, Monte Carlo is the wrong tool, and reporting a simulated value where a formula exists is a sign that something has gone wrong in the design rather than in the code.

The standard error also has a use beyond reporting: it is a diagnostic. A simulation whose standard error is not falling like $1/\sqrt{M}$ has a bug, a badly behaved payoff, or a random number generator with structure. Always compute it, and always compare it with the observed error on a case where the answer is known. Example 20.4 does both, on a calculation Chapter 9 needed.

Example 20.4 The exposure profile of Chapter 9

Chapter 9 valued counterparty risk on a five-year swap using an expected exposure profile it did not simulate. The calculation is a good one to meet first because the answer can be checked in closed form.

A dealer holds a five-year payer swap on $100 million, struck at the market so that it is worth

Wulin Suo: WSuo@Queensu.ca. For Personal Use, Do Not Distribute!

zero today. Take the swap rate to diffuse as a normal random walk, so that at a horizon t it stands at $S_t = S_0 + \sigma_r \sqrt{t}\, Z$ for a standard normal Z, with $\sigma_r = 1.456\%$ a year an absolute volatility rather than a proportional one. That is the normal-volatility convention of Section 19.1, which rates markets adopted because it permits a negative rate. It also makes the closed form below available: $V(t)$ is linear in the rate and so inherits its distribution. By equation (7.6) the swap is then worth

$$V(t) = L\,(S_t - K)\,A(t)$$

where L is the \$100 million notional, $K = S_0$ is the fixed rate, and $A(t)$ is the annuity of the $5 - t$ years still to run, which we take as $5 - t$ years of unit payments.

The simulation. Draw Z, form $V(t)$, keep $\max(V(t), 0)$, and average. Ten thousand paths at $t = 2.5$ give \$2.29 million, with a standard error of about \$0.03 million.

The closed form. What the simulation just estimated is the expected exposure $\mathrm{EE}(t) = E[\max(V(t), 0)]$: the average over states of what would be lost if the counterparty failed at t, with the states in which the dealer owes money contributing nothing. Because $V(t)$ is normal with mean zero, equation (9.3) evaluates that same expectation directly, with $\sigma_V(t)$ the standard deviation of the swap's value at that horizon:

$$\mathrm{EE}(t) = 0.3989\,\sigma_V(t) = 0.3989\,L\,\sigma_r\sqrt{t}\,A(t)$$

At mid-year points this gives, in millions, \$1.848, \$2.490, \$2.296, \$1.630 and \$0.616, the column Example 9.2 used; that example and Figure 9.1 explain the shape of the profile.

The closed form exists only because the swap's value is linear in a normal rate. Add an option, a barrier, or a second currency, and the expectation stops being available in closed form, and that is when the three-stage simulation of Section 9.1 stops being an illustration and becomes the only method.

20.7 DISCRETIZING THE PROCESS

Section 14.4 set out two ways to step a geometric Brownian motion forward and said the choice would return here.

The **Euler** scheme discretizes the stochastic differential equation directly:

$$S_{t+\Delta t} = S_t + \mu S_t \Delta t + \sigma S_t \varepsilon \sqrt{\Delta t} \tag{20.5}$$

The **exact** scheme uses the solution rather than the equation:

$$S_{t+\Delta t} = S_t \exp\left[\left(\mu - \frac{1}{2}\sigma^2\right)\Delta t + \sigma\varepsilon\sqrt{\Delta t}\right] \tag{20.6}$$

They are not equivalent, and the difference matters in two ways.

Equation (20.6) has no discretization error at all. It is the exact distribution of $S_{t+\Delta t}$ given S_t, so for geometric Brownian motion a single step from 0 to T is as accurate as a thousand. Any intermediate steps are needed only because the *payoff* looks at them: an Asian option needs the dates its average is taken on, a barrier option needs a fine grid.

Equation (20.5) can go negative. A large negative ε drives S below zero, which is impossible for the process it claims to simulate. With $\sigma\sqrt{\Delta t}$ small this is rare; with a large step or a high volatility it is not, and a simulation that produces negative prices will produce nonsense payoffs without necessarily producing an error message.

So use (20.6) whenever it exists, which for lognormal processes it always does. The reason Euler schemes are ubiquitous is that for most other processes (stochastic volatility, mean-reverting rates, jump models) no exact solution is available, and then the discretization error is real and must be managed.

A **predictor–corrector** step addresses one of its two parts, the error in the drift. Where the drift depends on the state being simulated, (20.5) freezes that drift at the value it has when the interval opens, and the step is biased whenever the state moves appreciably before the interval

Wulin Suo: WSuo@Queensu.ca. For Personal Use, Do Not Distribute!

closes. Write the process as

$$dX = a(X,t)\,dt + b(X,t)\,dW$$

Take the Euler step first, as a prediction,

$$\hat{X} = X_t + a(X_t,t)\,\Delta t + b(X_t,t)\,\varepsilon\sqrt{\Delta t} \tag{20.7}$$

then step again from X_t, averaging the drift at the two ends of the interval,

$$X_{t+\Delta t} = X_t + \frac{1}{2}\big[a(X_t,t) + a(\hat{X},t+\Delta t)\big]\Delta t + b(X_t,t)\,\varepsilon\sqrt{\Delta t} \tag{20.8}$$

The same ε stands in both, the draw of (20.7): drawing again would add a second independent source of noise where the intention was to remove a bias. The cost is one extra drift evaluation per step and no extra random numbers, and Example 20.5 measures what it buys.

Example 20.5 What a predictor–corrector corrects

Geometric Brownian motion lets the drift error be priced rather than simulated. Taking expectations through (20.5) multiplies S by $1 + \mu\Delta t$ over a step, the first two terms of $e^{\mu\Delta t}$, and through (20.8) by $1 + \mu\Delta t + \frac{1}{2}(\mu\Delta t)^2$, the first three. Each scheme therefore arrives at a different forward, and the Black–Scholes–Merton formula prices each of those forwards exactly.

Take $S_0 = K = 100$, $r = 10\%$, $\sigma = 20\%$ and $T = 1$. The exact forward is 110.5171 and the call is worth 13.2697; at twelve steps Euler reaches a forward of 110.4713 and the corrector 110.5170. The premiums those imply:

Steps	Euler	Error	Pred.–corr.	Error	Ratio
1	12.931946	0.337730	13.258455	0.011222	30
4	13.180623	0.089053	13.268935	0.000742	120
12	13.239625	0.030052	13.269593	0.000083	360
52	13.262708	0.006968	13.269672	0.0000045	1560

Halving the step halves the Euler error and quarters the corrector's: the drift is discretized to first order in Δt by (20.5) and to second order by (20.8).

One error is absent from the table. Pricing each scheme's forward under Black–Scholes isolates the drift, and an Euler simulation carries a second error on top: its terminal distribution is a product of normals rather than a lognormal, which is the same defect that lets (20.5) go negative. A predictor–corrector repairs the first and leaves the second alone.

The limit of the method is the distributional error just named. The diffusion term of (20.8) is the Euler one unchanged, so a quantity that depends on the fine structure of the path rather than on its expectation gains nothing from correcting the drift. Nor does the correction help a continuously monitored barrier, whose error comes from crossings missed between dates: that needs, on each step, the probability that the path crossed the barrier between the two simulated points, or Chapter 34's continuity correction.

That leaves a rule with two branches. Use the exact scheme where one exists. Use a predictor–corrector where the drift depends on the state and is not small beside the diffusion. Chapter 32's market model is where the predictor–corrector is normally applied, because there the drift of each rate is a function of all the others, and Example 32.4 shows it earning almost nothing, for the reason this section supplies: at a 3% rate the drift is about 1% of the diffusion, so correcting it cannot move the answer.

20.8 VARIANCE REDUCTION

Since the error falls like $1/\sqrt{M}$, buying accuracy with paths is expensive. Buying it with structure is cheap. Four techniques follow.

Wulin Suo: WSuo@Queensu.ca. For Personal Use, Do Not Distribute!

Antithetic variates. For every path generated with shocks $\varepsilon_1, \ldots, \varepsilon_n$, generate its mirror with $-\varepsilon_1, \ldots, -\varepsilon_n$ and average the two payoffs. Each path has the right distribution, and for a payoff monotone in the shocks the two are negatively correlated, so the variance of the average is lower. On the European put, the standard error at 100,000 paths (50,000 antithetic pairs) falls from 0.0159 to 0.0112, a factor of 1.4. It costs almost nothing and is worth doing for monotone payoffs, but it is not transformative, and for a payoff symmetric in the shocks, such as a straddle at the money, it is useless.

Control variates. Equation (20.1) returns in a more powerful form. Estimate a quantity Y whose true value is known alongside the target X, and report

$$\hat{X}_{\text{corrected}} = \hat{X} - \beta\left(\hat{Y} - Y^{\text{true}}\right), \qquad \beta = \frac{\text{Cov}(X, Y)}{\text{Var}(Y)} \tag{20.9}$$

With β chosen as in (20.9), the variance is reduced by a factor $1/(1-\rho^2)$, where ρ is the correlation between X and Y. Everything depends on ρ, and the difference between a good control variate and a mediocre one is enormous.

Example 20.6 A control variate that is worth 58 million paths

Price an arithmetic-average Asian call: $S_0 = K = 50$, $r = 5\%$, $\sigma = 30\%$, one year, twelve monthly fixings. There is no formula.

There is a formula for the *geometric*-average version, because the geometric average of lognormals is lognormal. It gives 4.012352 exactly. And the geometric and arithmetic averages of the same twelve prices are nearly the same number, so their payoffs are nearly perfectly correlated.

At 100,000 paths:

	Estimate	Standard error
Plain Monte Carlo	4.23399	0.02020
Geometric control variate	4.23671	0.00084

The measured correlation is $\rho = 0.99914$ and $\beta = 1.0485$. The variance falls by a factor of 583: to reach the same standard error, plain Monte Carlo would need 58 million paths.

The pattern that produced it is the one that gave the tree its accuracy in Example 20.1: find a problem as close as possible to the target and solvable in closed form, let the method price both, and correct the target by the error on the companion.

Importance and stratified sampling. Both put the paths where the answer is decided. A deep out-of-the-money option pays on a few paths in a thousand; **importance sampling** shifts the drift of the simulation toward the payoff region and reweights each path by how much likelier it was under the true drift than under the shifted one, which keeps the estimate unbiased, and it is the standard tool for deep out-of-the-money prices and for the rare tail losses of credit portfolios. **Stratified sampling** draws a fixed number of paths from each of several equal-probability bins of the normal draw that drives the path, so the sample cannot overweight a region by chance; moment matching, which rescales the draws to a sample mean of exactly 0 and variance of exactly 1, is a cruder relative, at the cost of a small bias.

Quasi-Monte Carlo. Replace pseudo-random numbers with a low-discrepancy sequence (Sobol or Halton) designed to fill the space more evenly than randomness does. Convergence can approach $1/M$ rather than $1/\sqrt{M}$ when a few of the draws decide most of the answer. The cost is that the usual standard error is no longer valid, since the points are not random; estimating the error requires several randomized repetitions. It works best when a few of the

Wulin Suo: WSuo@Queensu.ca. For Personal Use, Do Not Distribute!

random numbers drawn for each path decide most of its payoff, and paths are often built in a special order to make that so.

20.9 SIMULATING SEVERAL ASSETS

Monte Carlo's advantage over trees is dimension, and correlated assets are the reason to use it.

To simulate d assets with correlation matrix $\mathbf{R}$, factor $\mathbf{R} = \mathbf{L}\mathbf{L}^\top$ by Cholesky decomposition and set $\boldsymbol{\varepsilon} = \mathbf{L}\mathbf{z}$, where $\mathbf{z}$ is a vector of d independent standard normals. Here $\mathbf{L}$ is lower triangular (zero above the diagonal), and that shape is not incidental: it is what makes the construction sequential, with ε_1 built from z_1 alone, ε_2 from z_1 and z_2, and each shock in turn from the draws already used plus one new one. For two assets it reduces to

$$\varepsilon_1 = z_1, \qquad \varepsilon_2 = \rho z_1 + \sqrt{1-\rho^2}\, z_2$$

which is all that most problems need.

It is the correlation matrix that is factored, not the covariance matrix. Each asset's volatility enters separately, in its own price update, so the ε_i must come out with unit variance, which the two-asset form shows, since $\rho^2 + (1-\rho^2) = 1$. Factoring the covariance matrix instead returns shocks that already carry σ_i, and applying σ_i to them a second time is an error that produces plausible prices at the wrong volatility.

The factorization also needs an internally consistent matrix (technically positive semidefinite: one that gives no portfolio a negative variance), and one assembled pair by pair, from estimates over different windows or from traders' manual adjustments, often is not; Cholesky then fails partway through. Risk systems replace such a matrix with the nearest consistent one, by Higham's (2002) method.

Example 20.7 Valuing a relative-TSR performance award

Section 17.7 said that a performance award with a market condition is valued by simulation at grant and never trued up, and pointed here for the machinery.

The award: for each target unit, the executive receives shares at vesting in three years, in a quantity that scales linearly from 0% if the company's total shareholder return trails the index by 30% or more, through 100% if the two are equal, to a cap of 200% if the company beats the index by 30% or more. Settlement is in shares.

Take $S_0 = 50$, company volatility 35%, index volatility 20%, correlation 0.60, $r = 5\%$, three years. Simulating 400,000 correlated pairs gives:

Expected units earned, $E[u]$	0.8361
Expected share price, $E[S_T]$	58.1795
Naive product, $e^{-rT}E[u]\,E[S_T]$	41.8704
Correct value, $e^{-rT}E[u\,S_T]$	60.2468

Before reading anything off that table, apply the chapter's own rule to it. The exact $E[S_T]$ is $50e^{0.15} = 58.0917$, so the run came in 0.15% high, about one and a half standard errors, which is what 400,000 paths should deliver. The award value carries a standard error of roughly 0.13.

Two results follow.

The expected number of units is below one, at 0.836, even though the payout schedule is symmetric around a 100% payout. The company's higher volatility drags its median return below the index's (the $-\sigma^2/2$ term of Section 14.9), so the relative performance is more often slightly negative than slightly positive, while the 200% cap truncates the compensating upside.

And yet the award is worth 60.25 per unit, against a share price of 50. Expected units times expected share price gives 41.87, which understates the award by 30%; equivalently, the correct value is 44% higher. The gap is entirely the covariance: units are earned precisely on the paths where the share price is high, so $E[u\,S_T] \neq E[u]E[S_T]$.

Wulin Suo: WSuo@Queensu.ca. For Personal Use, Do Not Distribute!

Section 17.7's asymmetry now has a number attached: that 60.25 is expensed in full whether or not the award ever pays. Simulation is the natural method because the payoff, a capped and floored function of two correlated prices multiplied by one of them, is trivial to write inside a loop and awkward to integrate in closed form.

20.10 GREEKS FROM A SIMULATION

A simulated price is one number; a desk needs its sensitivities, because they are the coefficients of Chapter 18's expansion, equation (18.1), whose terms its hedges are built to cancel. The obvious approach, to revalue with S_0 bumped by a small h, difference the two prices, and divide by h, has a trap in it.

The trap is independent draws. If the bumped and unbumped simulations use independent random numbers, the difference of two estimates each carrying a standard error of, say, 0.016 has a standard error of 0.023, while the delta being measured might move the price by 0.05. The noise swamps the signal, and no number of paths fixes it efficiently. With common random numbers (the same draws reused in both runs) the two runs differ only through the bump, the noise largely cancels, and the estimator is usable.

This is the same principle as the control variate: the two runs share an error, and sharing it lets it cancel.

None of this difficulty arises on a lattice or a grid, which produce the value at every state and hand over delta, gamma and theta as differences of numbers already computed; Section 20.12 gives the formulas. Simulation is the method that has to work for them.

Two refinements remove the bump instead of taming it. **Pathwise differentiation** differentiates the payoff along each path, holding the draw fixed, which for a call is one application of the chain rule:

$$\frac{\partial}{\partial S_0}\, e^{-rT}(S_T - K)^+ = e^{-rT}\,\mathbf{1}\{S_T > K\}\,\frac{\partial S_T}{\partial S_0} = e^{-rT}\,\mathbf{1}\{S_T > K\}\,\frac{S_T}{S_0}$$

The second step is that $S_T = S_0 \exp[(r - \sigma^2/2)T + \sigma\sqrt{T}\,\varepsilon]$ is proportional to S_0 once ε is fixed, so $\partial S_T/\partial S_0 = S_T/S_0$. Average the right-hand side over the paths and the delta is in hand, from the paths that already produced the price, with no second simulation. The average is exactly unbiased here, because $e^{-rT}\mathbb{E}[S_T\mathbf{1}\{S_T > K\}] = S_0 N(d_1)$ makes the estimator's mean $N(d_1)$, the Black–Scholes delta itself. At $S_0 = K = 100$, $r = 10\%$, $\sigma = 20\%$ and $T = 1$ that delta is 0.7257, and 400,000 paths give 0.7262.

What this buys is not lower variance. Against a bump run on common random numbers the two variances agree (the ratio is 0.976 at $h = 1$ and 1.000 at $h = 0.01$), because as h falls the difference quotient becomes the derivative it approximates. What it buys is that there is no h to choose: the $h = 1$ bump reports 0.7344 against the exact 0.7257, an error of 0.0086 where the standard error is 0.0008. That error is bias. For a continuous payoff a small bump, or one taken on both sides of S_0, removes it at no cost in noise; the choice of h bites for a discontinuous payoff, where the bumped difference is nonzero on a vanishing fraction of paths and its variance grows like $1/h$.

It does require a differentiable payoff, and the failure is worse than it sounds. For a cash-or-nothing digital the payoff *is* the indicator, its derivative is zero wherever it exists, and the estimator returns zero on every path against a true delta of 0.0167: not a noisy answer but a confidently wrong one.

The **likelihood-ratio method** differentiates the density instead, which leaves the payoff untouched. Writing the price as $e^{-rT}\int \Pi(s)\,p(s;S_0)\,ds$, with Π the payoff and p the density

Wulin Suo: WSuo@Queensu.ca. For Personal Use, Do Not Distribute!

of S_T, and differentiating under the integral,

$$\frac{\partial}{\partial S_0}\, e^{-rT}\,\mathbb{E}[\Pi] = e^{-rT}\,\mathbb{E}\left[\Pi(S_T)\,\frac{\partial \ln p}{\partial S_0}\right] = e^{-rT}\,\mathbb{E}\left[\Pi(S_T)\,\frac{\varepsilon}{S_0\sigma\sqrt{T}}\right]$$

where the last step is a short calculation: $\ln S_T$ is normal with mean $\ln S_0 + (r - \sigma^2/2)T$, so $\partial \ln p/\partial S_0$ reduces to $(\ln S_T - \text{mean})/(\sigma^2 T S_0)$, and that is the draw ε divided by $S_0\sigma\sqrt{T}$. The estimator is therefore the discounted payoff times the draw, scaled: one multiplication on top of the price, for any payoff whatever. On the digital it returns 0.0166 against the exact 0.0167, with a standard error of 0.00004.

Nothing is free. Because the payoff is left whole and the derivative is pushed onto the draw, the estimator inherits the payoff's full variability: on the call it carries 8.6 times the variance of the pathwise estimator, which is 2.9 times the standard error and so about nine times the paths for the same precision. Dealers' pricing systems use pathwise where they can and likelihood ratio where they must. Applied to every input at once, pathwise differentiation becomes adjoint algorithmic differentiation (AAD). A simulated price is a chain of elementary steps (build the exponent, exponentiate, take the payoff, discount, average), and the chain rule can run along that chain in either direction. Run forward, it carries the derivative with respect to one input through every step, so n inputs cost n passes, no better than bumping. Run backward, it starts from the price and asks at each step how much the price moves per unit change in that step's result, a number called the step's **adjoint**; one backward pass then delivers the derivative with respect to every input at once. AAD software records the steps during the pricing run and replays them in reverse, so all the sensitivities together cost a fixed multiple of one price, typically three to five times, however many inputs there are; Section 32.13 shows what that does to the cost of a whole portfolio's sensitivities. The cost is memory, since every intermediate value must be kept for the backward pass, and the limit is pathwise differentiation's: a digital has no derivative to pass back. Example 20.8 runs it on one path.

Example 20.8 One path, run backward

Take the call above ($S_0 = K = 100$, $r = 10\%$, $\sigma = 20\%$, $T = 1$) and a single draw $\varepsilon = 0.5$. The forward pass computes and stores

$$x = \left(r - \frac{\sigma^2}{2}\right)T + \sigma\sqrt{T}\,\varepsilon = 0.18, \qquad S_T = S_0 e^{x} = 119.72, \qquad V = e^{-rT}(S_T - K)^{+} = 17.84$$

The backward pass, writing $\bar{y}$ for the adjoint of a quantity y, starts from $\bar{V} = 1$ and multiplies by one local derivative per step,

$$\bar{S}_T = e^{-rT}\,\mathbf{1}\{S_T > K\} = 0.9048, \qquad \bar{x} = \bar{S}_T\, S_T = 108.33$$

and each input reads its sensitivity off the step where it entered:

$$\frac{\partial V}{\partial S_0} = \bar{S}_T\,\frac{S_T}{S_0} = 1.083, \quad \frac{\partial V}{\partial \sigma} = \bar{x}\left(\sqrt{T}\,\varepsilon - \sigma T\right) = 32.50, \quad \frac{\partial V}{\partial r} = \bar{x}\,T - T\,V = 90.48, \quad \frac{\partial V}{\partial K} = -0.905$$

One backward pass has produced four sensitivities, each the pathwise estimate on this path; averaged over the paths they are delta, vega, rho and the sensitivity to the strike. Bump-and-revalue would have needed four more simulations.

20.11 AMERICAN MONTE CARLO

American Monte Carlo is the one genuinely difficult technique in the chapter. The problem, as Section 20.1 noted, is structural: a simulated path standing at an intermediate date does not know its own future, and an early-exercise decision requires exactly that.

The Longstaff–Schwartz method resolves it by *estimating* the continuation value with a

Wulin Suo: WSuo@Queensu.ca. For Personal Use, Do Not Distribute!

regression across paths.

Carry two numbers per path. Let C_i be the cash flow path i receives under the rule built so far, and τ_i the date it is received; initialize C_i to the payoff at expiration and τ_i to T. Now work backward through the exercise dates. At each date t_k:

1. Select the paths that are in the money at t_k. The decision is trivial elsewhere, and confining the fit to the region where it is close is what keeps the regression useful.

2. For each selected path form $Y_i = C_i e^{-r(\tau_i - t_k)}$: the value at t_k of the cash flow that path is currently scheduled to receive. Regress Y on a few functions of the current price, the regression's **basis functions** (1, S, S^2 will do), across those paths only.

3. Take the fitted value at each path's own S_{t_k} as its continuation value, and exercise where the immediate payoff exceeds it. On those paths overwrite C_i with the immediate payoff and τ_i with t_k, discarding the later cash flow: a path can be exercised once.

4. Step back to the previous date.

There is no regression at time zero. The estimate is the average of the discounted cash flows, $\frac{1}{M}\sum_i C_i e^{-r\tau_i}$, compared with the immediate payoff if the option may be exercised at once.

The regression is doing something specific and limited: it is estimating a conditional expectation from the cross-section of paths, using the fact that many paths pass near the same state. Example 20.9 runs the algorithm by hand.

Example 20.9 Longstaff–Schwartz on ten paths

Take the American put of Example 20.1 ($S_0 = K = 50$, $r = 5\%$, $\sigma = 30\%$, six months), with three exercise dates a sixth of a year apart and ten paths. Ten is far too few to price anything; it is enough to run the algorithm by hand.

Path	$t = 1/6$	$t = 2/6$	$t = 3/6$
1	58.83	55.93	62.73
2	47.62	56.43	57.46
3	43.85	54.54	46.88
4	42.21	43.86	45.49
5	51.71	47.49	48.36
6	48.92	47.20	46.39
7	64.47	71.06	70.94
8	40.90	41.89	49.10
9	55.01	49.11	61.99
10	42.38	39.52	38.54

At expiration $C_i = \max(50 - S_T, 0)$, which is nonzero on paths 3–6, 8 and 10.

At $t = 2/6$ six paths are in the money. Each carries $Y_i = C_i e^{-0.05/6}$, and the fitted regression is $\hat{Y} = 306.71 - 12.925\,S + 0.13674\,S^2$:

Path	S	Y	Continue	Exercise	Decision
4	43.86	4.4726	2.8718	6.1400	exercise
5	47.49	1.6264	1.2975	2.5100	exercise
6	47.20	3.5800	1.2909	2.8000	exercise
8	41.89	0.8925	5.2346	8.1100	exercise
9	49.11	0.0000	1.7579	0.8900	continue
10	39.52	11.3649	9.4837	10.4800	exercise

Path 9 is the instructive row: its own future cash flow is zero, so $Y_9 = 0$, yet the regression

Wulin Suo: WSuo@Queensu.ca. For Personal Use, Do Not Distribute!

assigns it a continuation value of 1.76 from what happened to the *other* paths near $S = 49$, and since that exceeds the 0.89 on offer, the rule continues. That is the idea: the continuation value is estimated from the cross-section, not read off the path's own future.

At $t = 1/6$ the cash flows just written are discounted back one more step, and the fit is $\hat{Y} = 375.53 - 15.604\,S + 0.16274\,S^2$:

Path	S	Y	Continue	Exercise	Decision
2	47.62	0.0000	1.5224	2.3800	exercise
3	43.85	3.0684	4.2281	6.1500	exercise
4	42.21	6.0890	6.8491	7.7900	exercise
6	48.92	2.7768	1.6620	1.0800	continue
8	40.90	8.0427	9.5716	9.1000	continue
10	42.38	10.3930	6.5367	7.6200	exercise

Path 10 shows what the method is and is not. It was scheduled to collect 10.48 at $t = 2/6$, worth 10.39 here, but the regression puts its continuation value at 6.54 and the immediate payoff is 7.62, so the rule exercises and the path collects 7.62. The rule is estimated, not clairvoyant, and on this path it gives up value that waiting would have earned.

Discounting each path's surviving cash flow from its own exercise date and averaging over all ten gives 3.6940. The converged value is 3.6970, and the agreement is luck: the European option priced on the same ten paths comes out at 2.46 against its true 3.5829, which is the honest measure of what ten paths are worth.

Three details decide whether an implementation works.

The regression needs far more paths than basis functions. With three basis functions and only three in-the-money paths the fit is exact, each fitted continuation value equals that path's own realized cash flow, and the exercise rule silently acquires perfect foresight, reporting a confident answer that is far too high. Keep the in-the-money count comfortably above the number of basis functions, or use fewer of them.

Scale the basis. At $S \approx 50$ the three columns hold values near 1, 50 and 2,500, and least squares on columns differing that much in scale loses precision. Replacing S by S/K throughout the basis makes the columns 1, S/K and $(S/K)^2$, all of order one near the strike, which removes the problem.

Two biases run in opposite directions. Fitting the regression and valuing the option on the same paths leaks a little foresight, which pushes the answer up. The suboptimal exercise rule pushes it down, and by more. Example 20.10 shows the net at path counts that matter. The low estimate also has a partner: Rogers (2002) and Haugh and Kogan (2004) showed independently that any exercise rule can also be turned into an *upper* bound on the value (the dual method), and Andersen and Broadie (2004) made that bound computable, so a least-squares price can be bracketed from both sides and the width of the bracket measures how good the rule is.

Example 20.10 Longstaff–Schwartz against the tree

The target is the American put of Example 20.1, worth 3.69702. With 25 or 50 exercise dates the method prices a **Bermudan** option, one exercisable only on a fixed set of dates, so that is its benchmark: on a tree that may exercise only on the same dates it is worth 3.6907 and 3.6938, below the American by 0.0063 and 0.0032. Using antithetic paths, with each pair counted once in the standard error, and a quadratic regression:

Wulin Suo: WSuo@Queensu.ca. For Personal Use, Do Not Distribute!

Paths	Dates	Estimate	Std. error	Bermudan	Error
20,000	25	3.69681	0.0175	3.6907	+0.0061
100,000	25	3.66563	0.0079	3.6907	−0.0251
100,000	50	3.68495	0.0077	3.6938	−0.0089
400,000	50	3.68705	0.0038	3.6938	−0.0068

In one run the rows scatter about their Bermudans by a standard error or more, and the first lands above; what is structural is the direction of the error on average, not its size in any row, because the regression produces an imperfect exercise rule and any rule other than the optimal one earns less. Adding paths shrinks the standard error but not the bias; a better regression basis reduces the bias, and more exercise dates move the target closer to the American.

A 200-step binomial tree gave this value to within 0.003 in a fraction of the time, so for a one-dimensional American option Longstaff–Schwartz is strictly the wrong choice. Its value is that it is the only method that works when the tree cannot be built: a callable note on a basket of five assets, a Bermudan swaption in a multi-factor model, or the exposure profiles of Chapter 9, where the early-exercise decision sits inside a simulation that must run anyway.

20.12 FINITE DIFFERENCE METHODS †

This section develops the third route to a price: solving the Black–Scholes–Merton partial differential equation numerically.

Write $V(S,t)$ for the price of the derivative being valued. Chapter 15 called it f; V is the usual symbol in this literature and is used throughout the section. Section 15.9 derived the form that carries a dividend yield,

$$\frac{\partial V}{\partial t} + (r-q)S\frac{\partial V}{\partial S} + \frac{1}{2}\sigma^2 S^2 \frac{\partial^2 V}{\partial S^2} = rV$$

Lay a grid over (S,t), replace each derivative by a difference quotient, and solve backward from the known terminal condition.

Three edges of the grid are filled in before the calculation can work backward from expiration, and they are what the scheme is solved from. The terminal column comes from the payoff. The two price boundaries come from what the option is worth when the underlying is worthless or very large: for an American put, $V(0,t) = K$, since it would be exercised at once, and $V(S_{\max},t) = 0$. A European put takes the discounted strike, $V(0,t) = Ke^{-r(T-t)}$, since it cannot be exercised before expiration. $S_{\max}$ has to sit far enough out that the option really is worthless there, and the scale to measure it against is $\sigma\sqrt{T}$, the standard deviation of $\ln S_T$: on the contract used below, an $S_{\max}$ three of these above $\ln S_0$ already makes the error from cutting the grid off less than 10^{-6}, and the examples' 200 (four times the strike) is more than six above it. Reaching far enough out is easy; the trap is that $S_{\max}$ also fixes $\Delta S = S_{\max}/M$, where M is now the number of price steps in the grid (not the path count of Section 20.6), and unless $S_0/\Delta S$ is a whole number the grid has no node at today's price at all. With $S_{\max} = 150$ and $M = 400$ the nearest node sits 0.125 below S_0, and the put comes back 0.053 too high, which is that 0.125 times the delta of −0.43 that Example 20.13 reports, and nothing to do with the scheme. The *coarser* $\Delta S = 0.5$, which does land on S_0, is out by 0.002. Choose M together with $S_{\max}$ so the grid has a node at S_0, or interpolate between the two that straddle it. Figure 20.5 shows the arrangement.

In the log variable $x = \ln S$ the coefficients stop depending on the state,

$$\frac{\partial V}{\partial t} + \nu\frac{\partial V}{\partial x} + \frac{1}{2}\sigma^2\frac{\partial^2 V}{\partial x^2} = rV, \qquad \nu = r - q - \frac{1}{2}\sigma^2$$

and that ν is the drift of (20.2). The examples below set $q = 0$, matching the trees they are

Wulin Suo: WSuo@Queensu.ca. For Personal Use, Do Not Distribute!

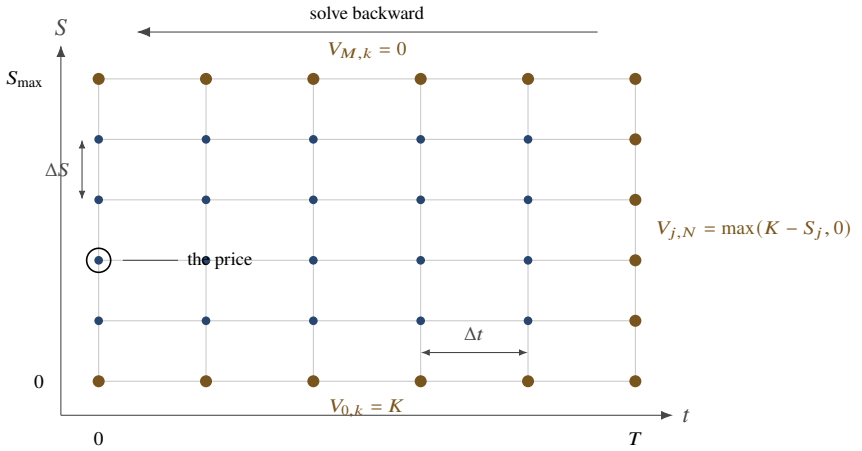


Figure 20.5: The grid for an American put. Larger (amber) nodes are boundary values; the rest are computed backward to $t = 0$.

measured against.

Three schemes differ in *when* the derivatives in S are evaluated. Write $V_{j,k}$ for the option's value at the grid point $S_j = j\,\Delta S$ and the date $t_k = k\,\Delta t$ (the first index counts price steps, the second counts dates), and $\mathbf{V}_k$ for the whole column of values at one date. The calculation runs backward, from the terminal column $\mathbf{V}_N$ to $\mathbf{V}_0$. Write $\mathcal{L}$ for the operator the equation puts against the time derivative,

$$\mathcal{L}V = rV - (r-q)S\frac{\partial V}{\partial S} - \frac{1}{2}\sigma^2 S^2 \frac{\partial^2 V}{\partial S^2}, \qquad \frac{\partial V}{\partial t} = \mathcal{L}V$$

Each derivative is then replaced by a difference over neighboring grid points:

$$\frac{\partial V}{\partial S} \approx \frac{V_{j+1,k} - V_{j-1,k}}{2\,\Delta S}, \qquad \frac{\partial^2 V}{\partial S^2} \approx \frac{V_{j+1,k} - 2V_{j,k} + V_{j-1,k}}{\Delta S^2}, \qquad \frac{\partial V}{\partial t} \approx \frac{V_{j,k+1} - V_{j,k}}{\Delta t}$$

The two space differences are centered on node j and carry an error of order ΔS^2; the time difference is one-sided and carries an error of order Δt. Substituting them, and using $S_j = j\,\Delta S$, makes every ΔS cancel, leaving three coefficients that depend on the node index alone:

$$(\mathcal{L}\mathbf{V}_k)_j = A_j V_{j-1,k} + B_j V_{j,k} + C_j V_{j+1,k}$$

$$A_j = \frac{1}{2}\left[j(r-q) - \sigma^2 j^2\right], \qquad B_j = r + \sigma^2 j^2, \qquad C_j = -\frac{1}{2}\left[j(r-q) + \sigma^2 j^2\right]$$

That the grid spacing has vanished is why the stability condition below is a statement about how many nodes there are rather than about how wide they are. With θ a weight between 0 and 1, the share of the operator taken at the earlier date, the scheme is

$$\frac{\mathbf{V}_{k+1} - \mathbf{V}_k}{\Delta t} = \theta\,\mathcal{L}\mathbf{V}_k + (1-\theta)\,\mathcal{L}\mathbf{V}_{k+1}$$

and gathering the unknown column on the left turns it into one linear system per step:

$$(\mathbf{I} + \theta\,\Delta t\,\mathcal{L})\,\mathbf{V}_k = (\mathbf{I} - (1-\theta)\,\Delta t\,\mathcal{L})\,\mathbf{V}_{k+1}$$

That equation is the method. Its right-hand side is known, its left-hand side is not, and each step delivers the column one date earlier. The three schemes are three readings of it, and Figure 20.6 draws what each costs.

Wulin Suo: WSuo@Queensu.ca. For Personal Use, Do Not Distribute!

Explicit, $\theta = 0$. The left side is $\mathbf{V}_k$ by itself, so nothing is solved and every interior node is read off directly:

$$V_{j,k} = a_j V_{j-1,k+1} + b_j V_{j,k+1} + c_j V_{j+1,k+1}, \qquad j = 1, \ldots, M-1$$

with $a_j = -\Delta t A_j$, $b_j = 1 - \Delta t B_j$ and $c_j = -\Delta t C_j$. Those three weights are the arrows in the figure's left panel, and on the log grid they approximate the probabilities of (20.2).

Implicit, $\theta = 1$. Now $(\mathbf{I} + \Delta t\, \mathcal{L})\mathbf{V}_k = \mathbf{V}_{k+1}$, which one row at a time reads

$$\Delta t A_j V_{j-1,k} + \left(1 + \Delta t B_j\right) V_{j,k} + \Delta t C_j V_{j+1,k} = V_{j,k+1}$$

for $j = 1, \ldots, M-1$. The first and last rows reach past that range, to $V_{0,k}$ and $V_{M,k}$, which are the boundary values of Figure 20.5 and therefore known, so they move across:

$$\begin{pmatrix} 1+\Delta t B_1 & \Delta t C_1 & & \\ \Delta t A_2 & 1+\Delta t B_2 & \ddots & \\ & \ddots & \ddots & \Delta t C_{M-2} \\ & & \Delta t A_{M-1} & 1+\Delta t B_{M-1} \end{pmatrix} \begin{pmatrix} V_{1,k} \\ V_{2,k} \\ \vdots \\ V_{M-1,k} \end{pmatrix} = \begin{pmatrix} V_{1,k+1} - \Delta t A_1 V_{0,k} \\ V_{2,k+1} \\ \vdots \\ V_{M-1,k+1} - \Delta t C_{M-1} V_{M,k} \end{pmatrix}$$

The matrix is tridiagonal, so it is never inverted. Write the row for $V_{j,k}$ as $\alpha_j V_{j-1,k} + \beta_j V_{j,k} + \gamma_j V_{j+1,k} = d_j$, with $\alpha_j = \Delta t A_j$, $\beta_j = 1 + \Delta t B_j$, $\gamma_j = \Delta t C_j$, and d_j the right-hand side above. The **Thomas algorithm** is Gaussian elimination that uses the three bands and touches nothing else: a forward pass clears the subdiagonal and a back substitution reads the column off. Starting from $\gamma'_0 = d'_0 = 0$, the forward pass runs for $j = 1, \ldots, M-1$:

$$\gamma'_j = \frac{\gamma_j}{\beta_j - \alpha_j \gamma'_{j-1}}, \qquad d'_j = \frac{d_j - \alpha_j d'_{j-1}}{\beta_j - \alpha_j \gamma'_{j-1}}$$

and the back substitution, starting from $V_{M-1,k} = d'_{M-1}$, runs back down for $j = M-2, \ldots, 1$:

$$V_{j,k} = d'_j - \gamma'_j V_{j+1,k}$$

Each pass is a single loop over M points, so a column costs $O(M)$ operations, the same order as an explicit step. And since the matrix is the same at every step, the γ'_j depend on nothing that changes: compute them once before the time loop begins, and each column costs only the d'_j recursion and the back substitution.

Crank–Nicolson, $\theta = 1/2$. The same system with half the operator on each side,

$$\left(\mathbf{I} + \frac{\Delta t}{2}\,\mathcal{L}\right)\mathbf{V}_k = \left(\mathbf{I} - \frac{\Delta t}{2}\,\mathcal{L}\right)\mathbf{V}_{k+1}$$

so a step costs the implicit scheme's solve plus one tridiagonal multiplication to form the right-hand side.

And, for an American option, one more line. Once a column has been computed, the early-exercise condition is imposed across it:

$$V_{j,k} \leftarrow \max\left(V_{j,k},\ K - S_j\right)$$

This is the free boundary Chapter 15 deferred, handled by projection. The boundary is never located; it is enforced wherever it binds, and where it does the node holds the payoff instead of the solved value. For the explicit scheme the projection is exact; after an implicit or Crank–Nicolson solve it carries an error of order Δt, which costs Crank–Nicolson its second

Wulin Suo: WSuo@Queensu.ca. For Personal Use, Do Not Distribute!

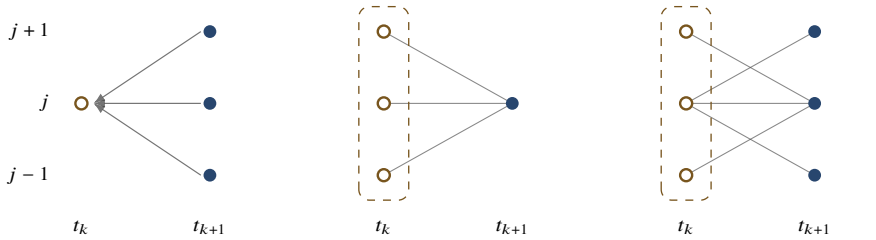


Figure 20.6: The nodes entering one equation under each weight. Filled nodes are known, hollow ones are being solved for, and t_k is the earlier date.

order. Imposing the condition inside the solve restores it, by Brennan and Schwartz's ordering of the elimination for a put or by projected iteration in general.

The weight is what sets the order in time. At $\theta = 1/2$ the right-hand side is the average of the operator at the two dates, which is its value at the midpoint between them to within $O(\Delta t^2)$, so the one-sided time difference above becomes a centered one about that midpoint. That is why Crank–Nicolson is second-order in time for a smooth problem where the other two are first-order, and Example 20.12 measures what survives of it on an American put.

The explicit scheme is a trinomial tree. At $\theta = 0$ each new value is an explicit weighted average of three neighbors, and on the log grid those weights are (20.2) to first order in Δt. The tree carries a $v^2\Delta t^2$ term the scheme drops, and discounts by $e^{-r\Delta t}$ where the scheme multiplies by $1 - r\Delta t$. Both differences fall by a factor of ten for every factor of ten in Δt, so Section 20.4's remark holds in the limit: the two methods are the same computation described in two vocabularies.

And it is only conditionally stable. The weights are probabilities only if Δt is small relative to $1/(\sigma^2 M^2)$, where M is the number of space steps. Violate that and the scheme does not merely lose accuracy: it explodes.

Example 20.11 What instability looks like

Solve the American put of Example 20.1 explicitly on a grid running to $S_{\max} = 200$:

Space steps	Time steps	Value
100	1,000	3.6766
200	1,000	beyond 10^{40}
200	4,000	3.6918

Refining the space grid while leaving the time grid alone destroyed the calculation. The middle row is not slightly wrong; it is dozens of orders of magnitude wrong, because the scheme amplifies rounding error geometrically once the stability condition fails, and its exact size depends on nothing more than the order of the arithmetic.

Doubling the space resolution therefore requires four times as many time steps. That is the practical objection to the explicit scheme, and it is why the implicit and Crank–Nicolson schemes exist: both are unconditionally stable, at the cost of solving a tridiagonal system at each time step, which the Thomas algorithm does in $O(M)$ operations.

There is one consolation. An unstable explicit scheme fails loudly, and a number beyond 10^{40} cannot be mistaken for a price. The failures worth fearing are the quiet ones, and Crank–Nicolson

Wulin Suo: WSuo@Queensu.ca. For Personal Use, Do Not Distribute!

has one: it damps error that flips sign from one node to the next only slowly, so the payoff's kink leaves oscillations near the strike that persist for many steps and show in gamma before the price. Rannacher's repair is to take the first few steps fully implicit.

Figure 20.7 draws the boundary that the three rows sit on either side of. It is a parabola, so the time steps a finer space grid needs rise with the square of its resolution: the middle row was not unlucky, only on the wrong side of the curve.

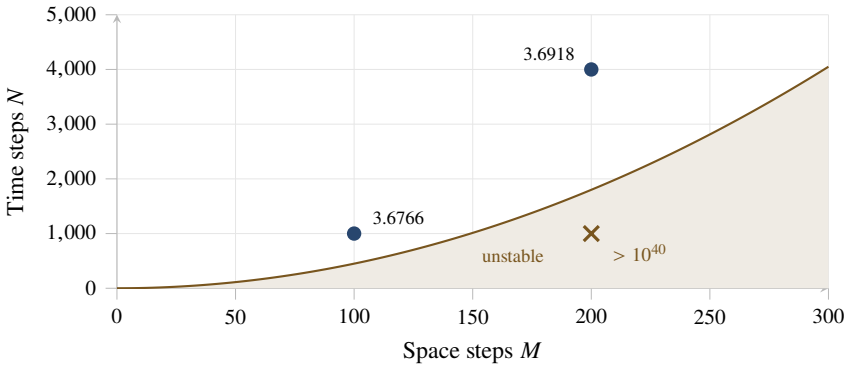


Figure 20.7: Stability of the explicit finite-difference scheme on the American put of Example 20.1. Below the curve $N = T\sigma^2 M^2$ the scheme diverges; the marks are the three grids of Example 20.11.

Example 20.12 Implicit against Crank–Nicolson

The same American put, benchmark 3.69702, gives:

Space	Time	Implicit	Error	Crank–Nicolson	Error
100	50	3.65776	−0.03926	3.67363	−0.02339
200	100	3.68196	−0.01506	3.69033	−0.00669
400	200	3.69056	−0.00646	3.69494	−0.00208
800	400	3.69403	−0.00299	3.69631	−0.00071

The implicit errors fall by roughly a factor of two per refinement, which is first order. The Crank–Nicolson errors fall by about three and a half, then three, then less: the symmetric time-averaging delivers second order on the smooth part of the problem, and the projection's first-order error shows through as the grid is refined. For nearly the same work Crank–Nicolson is more accurate at every grid size, and its lead grows with refinement, from under twice as accurate on the coarsest grid to four times on the finest, which is why it is the default.

Both converge from below, as the European put does on the same grids: the sign is the discretization's, not the exercise constraint's.

Why a desk uses finite differences at all. The method produces the value at every grid point, so the quantities a desk needs are already differences of numbers it holds. Writing j_0 for the node nearest the spot,

$$\Delta \approx \frac{V_{j_0+1,0} - V_{j_0-1,0}}{2\,\Delta S}, \qquad \Gamma \approx \frac{V_{j_0+1,0} - 2V_{j_0,0} + V_{j_0-1,0}}{\Delta S^2}, \qquad \Theta \approx \frac{V_{j_0,1} - V_{j_0,0}}{\Delta t}$$

which are the same three quotients the scheme is built from. A tree gives them as cheaply, from the nodes at steps one and two. For a book that must be revalued, and its risk recomputed, across a range of spot prices every evening, that is decisive.

Wulin Suo: WSuo@Queensu.ca. For Personal Use, Do Not Distribute!

Vega and rho do not come free: volatility and the rate are parameters of the equation rather than state variables, so each costs a second solve at a bumped input, which is why a desk needing a full set of Greeks reruns twice rather than five times.

Example 20.13 Greeks from the tree and from the grid

Ask each method for the Greeks of the American put of Example 20.1 as well as its price, at no extra cost in either case:

	Value	Δ	Γ	Θ
Tree, 6,000 steps	3.6969	−0.4300	0.03953	−3.187
Crank–Nicolson, 800×400	3.6963	−0.4299	0.03953	−3.190

Two methods sharing no arithmetic agree to three significant figures on all four numbers: Section 20.13's second habit applied to the Greeks, and the cheapest check available on an implementation.

The European option's closed-form delta and gamma are −0.4114 and 0.03668, against the American values above. The early-exercise feature shows more strongly in the Greeks than in the price: the value differs by 3%, the delta by 5%, and the gamma by 8%. A desk hedging an American book off European Greeks is wrong by more than the pricing error would suggest.

With a second solve, raising σ by one volatility point moves the price by 0.1373, and raising r by a basis point moves it by −0.0009.

20.13 CHOOSING A METHOD

The chapter reduces to a short list of decisions.

One state variable and early exercise → tree or finite difference. Use a control variate against the European, then extrapolate between step counts that share an offset. A few hundred steps suffice.

Path dependence → Monte Carlo, with a control variate. Look hard for a closely related closed form before accepting plain simulation; Example 20.6 is what that search is worth.

More than three state variables → Monte Carlo, necessarily. Grid methods are out of reach.

Both early exercise and high dimension → Longstaff–Schwartz, accepting the downward bias and reporting it.

Greeks across a grid of spot prices → finite difference, which delivers them as a by-product.

Desks divide along the same lines. Market makers in listed American equity options price on trees, fast enough to update quotes across every strike and expiration continuously; rates desks run one-factor callable and Bermudan books on trees or grids (Chapter 31) and simulate with regression only when the model has several factors (Chapter 32); XVA desks simulate a netting set's many state variables and take their sensitivities by AAD. Stochastic-volatility models are calibrated by transform pricing, and a market-condition award is valued for the accounts by simulation, as in Example 20.7.

Two habits matter more than the choice.

Always price something you already know. Run the European option through the same machinery and compare with the Black–Scholes–Merton formula. It costs almost nothing, it catches most implementation errors, and by (20.1) the discrepancy is not merely a diagnostic but a correction.

Always compute the answer twice. Two step counts, two path counts, two grid resolutions. A number computed once is not a number; it is a hypothesis. Between them the two habits catch most failures in this chapter: the instability of Example 20.11 falls to the second immediately, while the bias of Example 20.10, which a second path count cannot reveal, shows only against an independent answer, the Bermudan tree there or the dual upper bound where no tree exists.

Wulin Suo: WSuo@Queensu.ca. For Personal Use, Do Not Distribute!

20.14 WHERE THIS CAME FROM

Boyle (1977) introduced Monte Carlo to option pricing, but early exercise resisted simulation for two decades. Methods for it appeared through the 1990s, among them Broadie and Glasserman's (1997), and it was Longstaff and Schwartz's (2001) regression of the continuation value on the paths themselves that made American Monte Carlo routine, and that lets the exposure simulations of Chapter 9 price Bermudan options at all. The finite-difference route is older: Brennan and Schwartz (1977) priced the American put that way, and in 1978 showed the explicit scheme to be a lattice approximation of the diffusion, the identity Section 20.12 uses.

On lattices the pathologies came before the fixes. Boyle and Lau (1994) explained why a barrier priced on a tree converges badly and how node placement repairs it; Broadie, Glasserman and Kou (1997) gave the continuity correction for discretely monitored barriers that Chapter 34 checks against simulation. Barone-Adesi and Whaley (1987) supplied a fast analytic approximation to the American premium, which lattices and later methods now match in accuracy at little cost.

SUMMARY

A closed form requires a simple payoff, a simple process, and no early exercise; break one and a number must be computed. The pricing equation and the risk-neutral expectation are the same statement, so a price may be obtained backward by solving the equation or forward by simulating the expectation, and the two directions have opposite strengths: backward methods handle early exercise but fail in high dimension, forward methods handle path dependence and dimension but not early exercise.

Binomial trees converge at $O(1/n)$, with an error set by where the strike falls between two terminal nodes; at the money that is the parity of n. The control variate exploits it, correcting the target by the method's own error on a companion whose answer is known and cutting the American put's error by orders of magnitude; it removes what the two problems share and nothing else, so the companion should sit as close to the target as possible.

Trinomial trees do not converge faster. What they buy is an independent choice of Δx, which lets a node be placed on a barrier, and the explicit finite difference scheme is a trinomial tree in another notation. A known cash dividend destroys recombination (a proportional one, discrete or continuous, does not), and the remedy is to build the tree on the price net of the dividend's present value and add it back at each node.

Monte Carlo is unbiased with a standard error falling like $1/\sqrt{M}$, so a decimal place costs a hundredfold increase in paths, and structure beats brute force: a geometric-average control variate for an arithmetic Asian option removes so much variance that matching it with plain paths would take an impractical number of them. Lognormal paths are simulated exactly, and an Euler step, which can go negative, is a fallback for processes without a solution. Greeks from a simulation need common random numbers if bumped; the pathwise estimator needs a differentiable payoff and the likelihood-ratio estimator does not, and adjoint differentiation returns all sensitivities at once for the cost of a few prices. Correlated assets go through a Cholesky factor, and the reason to bother shows in a relative total shareholder return award, where multiplying expected units by expected price understates the value badly, because units are earned on precisely the paths where the share price is high. Longstaff–Schwartz regresses the continuation value on the cross-section of in-the-money paths and is biased downward, since a suboptimal exercise rule earns less; a dual calculation supplies the matching upper bound, and the method is the right choice only where a tree cannot be built.

Finite difference schemes differ in when the derivatives in S are evaluated. The explicit scheme is only conditionally stable, while implicit and Crank–Nicolson are unconditionally stable, and Crank–Nicolson is second-order in time because its weight centers the time

Wulin Suo: WSuo@Queensu.ca. For Personal Use, Do Not Distribute!

difference, an order that early exercise imposed after the solve gives back. A grid returns delta, gamma and theta as differences of values it has already computed; vega and rho need a second solve. Two habits govern everything: price something whose answer is already known, and compute every answer twice.

FURTHER READING

Glasserman's *Monte Carlo Methods in Financial Engineering* is the standard reference, and the one that explains why variance reduction works rather than only how to implement it. Longstaff and Schwartz (2001) is the original least-squares Monte Carlo paper and is short; the subsequent literature on the method's bias and on choosing regression bases is large and worth sampling before implementing it. Wilmott, Howison and Dewynne treat the partial differential equation route with the numerical analysis intact, which is the place to go for stability conditions derived rather than quoted. Capriotti (2011) is the practitioner account of adjoint differentiation for Greeks. On trees, Broadie and Detemple (1996) smooth the terminal payoff by putting the Black–Scholes–Merton value at the last step, and Boyle and Lau (1994) show how node placement interacts with a barrier; between them they explain a family of convergence pathologies that Chapter 34 meets again. For low-discrepancy sequences, the practical literature diverges sharply from the theoretical bounds, and any implementation should be tested against a known answer at the dimensions actually used.

PRACTICE QUESTIONS

20.1 A binomial tree gives 3.5724 at 100 steps and 3.5777 at 200. Estimate the exact value by Richardson extrapolation, state what you must check before trusting it, and explain why the binomial error alternates in sign with the parity of n when $S_0 = K$.

20.2 A tree gives an American put value of 3.6914 and a European put value of 3.5724 at the same step count, and the Black–Scholes–Merton formula gives 3.5829 for the European. Compute the control-variate corrected American value, and explain why the correction helps enormously at odd step counts and only slightly at even ones.

20.3 State two things a trinomial tree buys over a binomial tree and one thing it does not, and name the finite difference scheme that is a trinomial tree in another notation.

20.4 Explain why a known cash dividend destroys the recombination of a binomial tree, and describe the standard remedy.

20.5 A Monte Carlo estimate has a standard error of 0.04 at 10,000 paths. How many paths are needed for a standard error of 0.005?

20.6 Explain why the exact lognormal simulation step is preferable to an Euler step, giving both reasons.

20.7 A control variate has correlation 0.95 with the target. By what factor is the variance reduced? Compare with the 0.99914 achieved by the geometric Asian control variate and explain why the improvement is so much larger than the correlations differ.

20.8 Explain why bumping the spot price and revaluing gives a poor delta if the two simulations use independent random numbers.

20.9 State the direction of the Longstaff–Schwartz bias, explain why it goes that way, and state when the method is nonetheless the right choice.

20.10 Explain why the explicit finite difference scheme becomes unstable when the space grid is refined, and state how the time grid must respond.

20.11 Explain why a relative total shareholder return award cannot be valued as expected units

Wulin Suo: WSuo@Queensu.ca. For Personal Use, Do Not Distribute!

times expected share price, and state the direction and rough size of the error.

FURTHER QUESTIONS

20.12 Richardson extrapolation can return an answer worse than either of the values it is built from. Derive the condition on the error sequence under which extrapolation improves the estimate, explain how the offset of Section 20.3 decides whether two step counts satisfy it, and propose a diagnostic a pricing system could run automatically to detect a violation before reporting a price.

20.13 Example 20.1 argues that a control variate removes the error the two problems share and nothing else. Formalize this: write the tree's American and European errors as a shared component plus an idiosyncratic one, show what (20.1) does to each, and use the result to predict how the correction's effectiveness changes as the option moves deep into or out of the money.

20.14 The Asian control variate of Example 20.6 achieves a variance reduction of 583 through a correlation of 0.99914. Explain why the geometric and arithmetic averages are so highly correlated, determine how the correlation and hence the reduction would change if the number of fixings rose from 12 to 250, and discuss whether the closed form for the geometric case remains available.

20.15 Example 20.7 finds that the naive product of expected units and expected share price understates the award's value by 30%, so that the correct value is 44% higher. Decompose the correct value into the naive product plus a covariance term, explain the economic content of that term, and state what it implies about a company that grants relative total shareholder return awards and simultaneously reports their value using an expected-payout methodology.

20.16 Section 20.11 states that Longstaff–Schwartz is biased low and that adding paths does not remove the bias. Explain why the bias is not a small-sample effect, describe how one could construct an *upper*-biased estimate to bracket the true value, and discuss what a risk manager should report when only the lower estimate is available.

20.17 Example 20.11 shows an explicit scheme returning a number beyond 10^{40}. Derive the stability condition for the scheme, show that it is equivalent to requiring the three trinomial weights of (20.2) to be nonnegative, and explain why this equivalence means the instability is a statement about probabilities rather than about arithmetic.

Wulin Suo: WSuo@Queensu.ca. For Personal Use, Do Not Distribute!